Butterfly Fresh Ride
Complete Project File Guide

A detailed inventory of the tracked source, configuration, database, and asset files

Project: Next.js storefront and admin application
Inventory scope: 104 Git-tracked files

How to read this guide

Files are grouped by their role in the application. Each entry describes the file’s purpose and, where useful, how it connects to other parts of the site. The inventory covers tracked repository files only. Dependencies (node_modules), generated Next.js output (.next), build caches, and the ignored local .env.local file are excluded. The environment template is included, but it contains placeholders rather than private keys.

Architecture at a glance

The project is a Next.js App Router application written in TypeScript. Its public pages render a marketing storefront and product catalogue; browser components provide interactions; Supabase supplies authentication and PostgreSQL data; Zustand persists the shopping cart; and API route handlers mediate order, coupon, enquiry, product-admin, and payment operations. Tailwind utilities and shared CSS classes provide styling, while Framer Motion and React Three Fiber supply animation and a 3D hero visual.

Area	Responsibility
Storefront and data	Server pages read products/content from Supabase; interactive components handle selection, forms, and cart actions in the browser.
Cart and checkout	Product controls write to the persisted Zustand cart. Checkout posts cart details and totals to the orders API; the current UI exposes COD even though Razorpay API support exists.
Accounts and admin	Supabase Auth handles customer access. The admin layout checks profiles.is_admin; admin product mutations repeat authorization checks in their API routes.
B2B	The enquiry form validates fields in the browser, posts to the enquiries API, and the admin enquiry page reads the saved records.

Important implementation notes

• There are two product detail route families: /product/[slug] and /products/[slug]. The homepage uses the singular path while catalogue cards use the plural path and a different component/data presentation.

• Razorpay order creation and signature verification exist, but the checkout UI currently renders a COD-only payment control.

• The account dashboard links to /account/addresses, password reset redirects to /account/update-password, and the footer links to policy routes; these routes do not have corresponding tracked page files here.

• The diagnostic /api/debug-products route returns project configuration/data diagnostics and should be considered when deciding what operational endpoints should be publicly reachable.

• Several server pages use fallback data or catch backend errors; this improves development resilience but can obscure configuration or database problems.

1. Project configuration and metadata

.env.example

Template for local configuration. Lists the public Supabase URL and anon key, Razorpay public/server credentials, and site URL. Copy it to .env.local and replace placeholders; never put real credentials in this tracked template.

.gitignore

Tells Git to omit node_modules, Next.js/build output, local environment files, logs, editor state, and TypeScript build metadata. This protects local configuration and keeps generated files out of source control.

AGENTS.md

Repository-specific instructions for coding agents. It notes that this project's installed Next.js version has changed APIs and directs contributors to consult its bundled documentation before making code changes.

README.md

Human-facing project documentation: what the storefront is, setup steps, environment configuration, framework/services, advertised routes, database migration setup, admin promotion SQL, brand colours, and sample coupon data.

package.json

Defines the package name/version, npm scripts (dev, build, start, lint), and direct runtime/development dependencies. It identifies Next.js/React, Supabase, Razorpay, Zustand, Tailwind, Zod, animation, 3D, and form libraries used by the app.

package-lock.json

Locks the full npm dependency graph to exact resolved versions and integrity hashes so installs are reproducible. It is generated/maintained by npm rather than edited as application logic.

next.config.ts

Configures Next.js remote image hosts for Supabase Storage and Cloudinary. It also opts out of build-time TypeScript and ESLint failure, meaning a passing build alone does not establish that either check is clean.

eslint.config.mjs

Configures ESLint with Next.js Core Web Vitals and TypeScript rule sets and excludes generated build folders and Next.js type output.

postcss.config.js

Registers Tailwind CSS and Autoprefixer as PostCSS plugins, allowing Tailwind utilities and vendor-prefixed styles to be generated during CSS processing.

tailwind.config.ts

Defines Tailwind's source scan paths and the design tokens used across the UI: brand colours, Inter/Playfair Display fonts, green gradient, and reusable animation names/keyframes.

tsconfig.json

Sets strict TypeScript compilation, browser/modern JS libraries, bundler module resolution, React JSX, incremental checking, the Next plugin, and the @/* alias rooted at the repository.

proxy.ts

Exports a Next request proxy that currently lets requests pass through. Comments explain that authentication is handled in the account/admin server pages; this file does not itself protect routes.

2. App shell and customer routes

app/favicon.ico

The site icon served by browsers and shown in browser tabs/bookmarks.

app/globals.css

Global CSS entry point. Applies Tailwind layers, base resets and typography, shared containers/section spacing, brand button classes, gradients, and shimmer animation.

app/layout.tsx

Root App Router layout. Imports global styles, declares viewport and SEO/social metadata, loads Google Fonts, and wraps every route in the shared HTML/body shell.

app/page.tsx

Homepage server component. Attempts to load the Fresh Ride product variants, active FAQs, and testimonials from Supabase; uses embedded fallback content if the backend is unavailable, then composes the marketing sections between Navbar and Footer.

app/products/page.tsx

Product catalogue route. Loads products, images, prices, categories, and feature tags from Supabase, reports query/empty-state messages, and renders the filterable ProductsGrid.

app/products/[slug]/page.tsx

Plural product-detail route used by the catalogue links. Loads a product by slug, calculates the lowest listed price, orders images and active variants, and renders descriptive content plus AddToCartSection; an unknown slug returns not-found.

app/product/[slug]/page.tsx

Singular product-detail route used for the main Fresh Ride kit. Fetches a product and related variants/images, with built-in demo/fallback data, then renders the richer gallery/detail client component.

app/product/[slug]/ProductDetailClient.tsx

Interactive detail-page UI: breadcrumb, image gallery and thumbnails, rating/price display, variant and quantity selectors, stock messaging, add/buy actions, trust marks, product specifications, and shipping/returns information.

app/shop/page.tsx

Server-rendered shop route. Reads the main kit and active pack variants from Supabase, falls back to local pack data, and passes the result to ShopClient.

app/shop/ShopClient.tsx

Interactive pack listing. Supports pack-size and price sorting, computes discounts and per-kit price, shows stock, and adds a selected pack to the cart or routes to checkout.

app/cart/page.tsx

Cart screen backed by the persistent Zustand store. Lets shoppers edit quantities/remove lines, apply or remove coupon codes through the API, and review subtotal, discount, shipping, and total before checkout.

app/checkout/page.tsx

Checkout form for contact and delivery details. Zod-backed React Hook Form validation protects the UI; submission sends the cart and calculated totals to /api/orders. The visible payment choice is currently Cash on Delivery.

app/orders/[id]/page.tsx

Order confirmation page. Reads the payment query parameter to show COD or paid wording, displays the order ID, next steps, and links to continue shopping or view account orders.

app/business/page.tsx

B2B/wholesale landing page. Adds an introductory banner and reuses the homepage B2B section, including its enquiry form.

app/contact/page.tsx

Static contact page with phone, email, address, and a call-to-action linking visitors to the business enquiry form.

app/account/page.tsx

Authenticated account dashboard. Redirects signed-out visitors to login; shows profile/order/address entry points, displays a confirmation notice when applicable, conditionally links admins to the admin panel, and provides sign-out.

app/account/login/page.tsx

Client-side Supabase email/password sign-in page. Uses the shared login schema, reports authentication errors, and links to account creation and password reset.

app/account/register/page.tsx

Client-side account registration. Validates user details, creates a Supabase account with name/phone metadata, directs confirmation email back to the app, and offers a resend-confirmation action.

app/account/reset-password/page.tsx

Requests a Supabase password-reset email and presents a success state or error message. Its configured redirect targets /account/update-password, for which there is no tracked page in this repository.

app/account/profile/page.tsx

Requires an authenticated user, reads their profile values (falling back to Auth metadata), and renders ProfileEditor with the current name, phone, email, and user ID.

app/account/orders/page.tsx

Loads orders belonging to the signed-in user and displays order number, date, total, status, item count, and tracking number when available.

app/auth/confirm/route.ts

Supabase email-confirmation callback. Exchanges an authorization code or verifies an email token hash, then redirects to the account page on success or login with an error indicator.

3. Admin pages and API endpoints

app/admin/layout.tsx

Shared admin authorization boundary. Requires a Supabase user and a profile with is_admin=true, otherwise redirects to login/account; authorized pages receive the AdminSidebar layout.

app/admin/page.tsx

Admin dashboard. Fetches recent orders, product data, customer count, and new-enquiry count, then summarizes revenue/status and shows quick links.

app/admin/products/page.tsx

Product-management table showing image, name/slug, first-variant price and stock, visibility, and edit/hide/delete actions.

app/admin/products/new/page.tsx

Admin page that hosts ProductForm in create mode.

app/admin/products/[id]/page.tsx

Loads one product with variants/images by ID and passes it to ProductForm in edit mode; unknown IDs return not-found.

app/admin/orders/page.tsx

Admin order overview with order totals, payment status, customer and shipping details, notes, line items, and summary counts for pending/delivered/paid revenue.

app/admin/customers/page.tsx

Admin table of profile records, showing customer name, phone, whether the profile is an admin, and signup date.

app/admin/enquiries/page.tsx

Admin business-enquiry inbox. Shows company/contact details, type, location, requested quantity, requirements, status, date, and counts by status.

app/admin/content/page.tsx

Read-only overview of FAQ and testimonial records, including activity, sort order, ratings, and whether a testimonial is demo content.

app/api/orders/route.ts

Order API: POST creates an order and line items, confirms COD orders, or creates a Razorpay order for other methods. GET requires sign-in and returns the user's orders or all orders for admins.

app/api/payment/verify/route.ts

Payment verification API. Checks Razorpay's HMAC signature and then marks the associated payment and order as paid/confirmed.

app/api/cart/coupon/route.ts

Coupon validation API. Checks active status, expiration, minimum order, and usage limit, then calculates either a percentage or fixed discount.

app/api/enquiries/route.ts

Business enquiry API. POST inserts submitted enquiry data. GET retrieves enquiries for the admin view. The current POST catch branch returns success even if storage failed, so success may not mean the enquiry was saved.

app/api/admin/products/route.ts

Admin-only product creation endpoint. Checks the authenticated user's admin profile, inserts product details and an initial variant, and optionally inserts a primary image.

app/api/admin/products/[id]/route.ts

Admin-only product mutation endpoint. PATCH updates product data and its first variant (or inserts a variant if absent); DELETE removes the product.

app/api/debug-products/route.ts

Dynamic diagnostic endpoint returning the configured public Supabase URL, product count/slugs, query error, and current timestamp. It is for troubleshooting product-data configuration.

4. Reusable UI components

components/layout/Navbar.tsx

Responsive public-site navigation with scroll-sensitive appearance, mobile drawer, product/business links, phone link, account name lookup through Supabase Auth/profile, and cart item badge.

components/layout/Footer.tsx

Shared footer containing brand description, social links, product/business navigation, contact details, and links to policy routes.

components/account/ProfileEditor.tsx

Editable profile form. Validates name and Indian mobile number, updates both the profiles row and Supabase Auth metadata, shows errors, and refreshes server-rendered data.

components/account/SignOutButton.tsx

Signs out through the browser Supabase client, then navigates home and refreshes the route.

components/admin/AdminSidebar.tsx

Fixed admin navigation panel with active-route styling, logo, links to admin sections, storefront link, and account link.

components/admin/ProductForm.tsx

Reusable product create/edit form. Edits descriptions, slug, category, feature tags, first-variant price/stock, image path, and visibility, then calls the appropriate admin API.

components/admin/DeleteProductButton.tsx

Prompts for confirmation, calls the product DELETE API, and refreshes the listing. It currently does not display a failed request response to the user.

components/admin/ToggleActiveButton.tsx

Calls the product PATCH API to switch visibility between active and hidden, then refreshes the page.

components/home/HeroSection.tsx

Homepage hero with lifestyle background, marketing copy, animated calls to action, and a desktop-only 3D display. It retains a commented-out earlier hero implementation.

components/home/HeroScene3D.tsx

React Three Fiber scene imported by the active homepage hero. Renders three layered bedding pieces with lighting, float motion, pointer tilt, and hover-spread animation.

components/3d/Hero3DScene.tsx

Alternative 3D sleeper-berth scene: mattress/frame, sheet, pillow, blanket, ambient particles, lights, and environment. It is not the scene imported by the active homepage hero.

components/home/TrustStrip.tsx

Four-item highlight strip emphasizing kit contents, bulk supply, single-use convenience, and business support.

components/home/ProductIntroSection.tsx

Product introduction with the three-piece kit image, contents list, explanatory text, and links to the shop and product specifications.

components/home/WhyChooseSection.tsx

Animated benefits grid describing convenience, kit completeness, target settings, portability, applications, and bulk supply.

components/home/UseCasesSection.tsx

Image-card grid for sleeper buses, hotels, healthcare, travel operators, wellness, and home use.

components/home/HowToUseSection.tsx

Five-step visual guide: open the pack, lay out the sheet, prepare the pillow, rest, and dispose responsibly.

components/home/ProductPackages.tsx

Homepage pack/pricing cards with discount and stock indicators plus add-to-cart and buy-now actions.

components/home/B2BSection.tsx

Business-focused marketing and enquiry form. Uses React Hook Form/Zod validation and posts to /api/enquiries; shows a confirmation screen after an HTTP-success response.

components/home/TestimonialCarousel.tsx

Animated customer quote carousel with star ratings, manual arrows/dots, and timed rotation that pauses while hovered.

components/home/FAQSection.tsx

Accessible expandable FAQ accordion. Receives FAQ rows as props and animates the open answer.

components/home/FinalCTASection.tsx

Closing homepage call-to-action section linking shoppers to the shop and visitors to contact details.

components/products/ProductsGrid.tsx

Filterable catalogue grid. Product cards display primary image, category, feature tags, price/coming-soon status, and a detail link; exports a skeleton component for loading UI.

components/products/AddToCartSection.tsx

Variant/quantity selection and add-to-cart control for catalogue product details. Disables unavailable/price-pending products and provides a WhatsApp enquiry link for price-pending products.

5. Shared application logic and types

hooks/useCartStore.ts

Zustand cart store persisted under butterfly-cart in browser storage. Implements add/remove/update/clear, coupon state, stock-limited quantity, item count, subtotal, shipping threshold, and total calculations.

lib/supabase/client.ts

Creates the browser Supabase client using the public URL and anon key for interactive components and user authentication.

lib/supabase/server.ts

Creates a cookie-aware server Supabase client for App Router server components and API routes, forwarding cookie reads/writes through Next’s headers API.

lib/utils/helpers.ts

Shared helpers for INR formatting, generated order numbers, percentage discount display, slug conversion, text truncation, Indian state options, and WhatsApp URLs.

lib/validations/schemas.ts

Zod schemas and inferred TypeScript types for checkout, business enquiries, login, and registration, including email, Indian phone, PIN code, and matching-password validation.

types/database.ts

Application-side Supabase schema contract. Defines JSON, row/insert/update shapes for the database tables, convenience row aliases, and joined product/cart/order types.

6. Database migrations

supabase/migrations/001_initial.sql

Initial Supabase schema and seed migration. Creates profiles, products, variants, images, addresses, carts, cart items, orders, order items, payments, coupons, testimonials, FAQs, and business enquiries; adds the signup profile trigger, RLS policies, and initial kit/FAQ/testimonial/coupon data.

supabase/migrations/002_order_visibility.sql

Enables row-level security on order_items and adds policies allowing customers to read their own order lines and admins to access order lines.

supabase/migrations/003_additional_products.sql

Adds category and feature_tags to products, seeds eight accessory products with inactive placeholder variants, and adds primary local image paths when not already present.

7. Public images and static assets

public/images/logo.png

Butterfly brand logo used in the public navigation/footer, account screens, and admin sidebar.

public/images/lifestyle/b2b_fleet.png

Fleet/business lifestyle visual used in the B2B section and in multiple use-case cards.

public/images/lifestyle/hero_bg.jpg

Large background photograph used behind the homepage hero text and the site’s social preview metadata.

public/images/lifestyle/hero_sleeper_berth.png

Sleeper-berth lifestyle photograph used in the kit instructions and fallback product gallery.

public/images/lifestyle/hotel_room.png

Hotel-room lifestyle image used for accommodation and wellness use cases.

public/images/lifestyle/howto_unpack.png

Image illustrating unpacking/using the bedding kit in the instructional section.

public/images/lifestyle/sleeper_bus.png

Sleeper-bus lifestyle visual used by the use-case cards.

public/images/products/kit_flatlay.jpg

Flat-lay photograph of the kit; retained in the public product asset collection.

public/images/products/three_piece_kit.png

Primary three-piece kit image used in the product-introduction section and fallback product/shop content.

public/products/compact-travel-blanket.jpg

Accessory product image for the compact travel blanket seeded by migration 003.

public/products/disposable-foam-ear-plugs.jpg

Accessory product image for disposable foam ear plugs.

public/products/disposable-pillow-cover.jpg

Accessory product image for disposable pillow covers.

public/products/disposable-travel-slippers.jpg

Accessory product image for disposable travel slippers.

public/products/inflatable-neck-pillow.jpg

Accessory product image for the inflatable travel neck pillow.

public/products/reusable-silicone-ear-plugs.jpg

Accessory product image for reusable silicone ear plugs.

public/products/sleep-eye-mask.jpg

Accessory product image for the sleep eye mask.

public/products/travel-hygiene-kit.jpg

Accessory product image for the travel hygiene kit.

public/file.svg

Generic Next.js starter SVG illustration; not part of the main storefront product flow.

public/globe.svg

Generic starter globe illustration; available as a static public asset.

public/next.svg

Next.js logo SVG retained from the starter template.

public/vercel.svg

Vercel logo SVG retained from the starter template.

public/window.svg

Generic starter window SVG illustration.

File-count note

This guide was prepared against the repository's tracked-file inventory. Generated and installed files are intentionally outside that count because they are produced by the toolchain and are not maintained as project source.